UNIVERSITÀ POLITECNICA DELLE MARCHE

Facoltà di Ingegneria

Corso di laurea magistrale in
Ingegneria Informatica e dell'Automazione

Data Science



Prompt Engineering

Realizzazione di una Dashboard sul monitoraggio di una serra intelligente e di un Simulatore di Epidemie

Studenti:
Diocis Daniel
Ricci Ettore
Salvatore Pietro
Sebastianelli Nicola

Anno Accademico 2025/2026

Indice

1 Monitoraggio di una GreenHouse **2**
- 1.1 Descrizione delle dashboard . . . 2
- 1.2 Diagramma E-R . . . 2
- 1.3 Scelta del modello . . . 4

2 Generazione delle Dashboard **4**
- 2.1 Prompt0: Diagramma E-R . . . 4
- 2.2 Prompt1: Generazione prima Dashboard . . . 5
- 2.3 Prompt2: Modifiche estetiche . . . 5
- 2.4 Prompt3: Correzione numero sensori . . . 6
- 2.5 Prompt4: Ricorrezione numero sensori e correzione grafici . . . 7
- 2.6 Prompt5: Modifica grafici . . . 8
- 2.7 Prompt6: Modifica grafico a torta . . . 9
- 2.8 Prompt7: Ripristino grafico letture ambientali . . . 10
- 2.9 Dashboard 2 . . . 11
  - 2.9.1 Prompt1: . . . 11
  - 2.9.2 Prompt2 Dashboard 2 . . . 13

3 Simulatore di Epidemie **15**
- 3.1 Descrizione del software . . . 15
- 3.2 Scelta del modello . . . 16

4 Processo di generazione **16**
- 4.1 Prompt1: Simulatore base . . . 17
- 4.2 Prompt2: Migliorare la GUI . . . 19
- 4.3 Prompt3: Grafico dell'epidemia . . . 21
- 4.4 Prompt4: Aggiungere mortalità . . . 22
- 4.5 Prompt5: Vaccinazione . . . 23
- 4.6 Prompt6: Cure . . . 25
- 4.7 Prompt7: Controlli della simulazione . . . 26
- 4.8 Prompt8: Statistiche finali . . . 27
- 4.9 Prompt9: Migliorare l'animazione . . . 29
- 4.10 Prompt10: Dashboard finale . . . 30
- 4.11 Prompt11: Correzioni finali . . . 31

5 Risultati **32**
- 5.1 Dashboard . . . 32
- 5.2 Software . . . 32

1 Monitoraggio di una GreenHouse

Il progetto consiste nella generazione dinamica di intefacce per la Business Intelligence, in particolare nella creazione di dashboard operative per una Smart Greenhouse. Partendo da un diagramma E-R, l'IA automatizza il design di interfacce grafiche complesse, garantendo la coerenza tra i KPI tecnici e rappresentazioni intuitive. L'obiettivo principale è dimostrare come il linguaggio naturale possa sostituire la progettazione manuale del front-end, permettendo di passare istantaneamente dalla logica dei dati a una visualizzazione interattiva e professionale ottimizzata per il monitoraggio di una serra idroponica intelligente.

1.1 Descrizione delle dashboard

Le dashboard generate per questo progetto sono due, la prima è stata progettata per chi deve garantire che l'infrastruttura funzioni senza problemi, concentrandosi sull'efficienza della rete e sull'andamento delle risorse utilizzate. La seconda interfaccia sposta l'attenzione sulle piante, permettendo di capire a colpo d'occhio la salute di esse. Sono dashboard pensate sia per manutentori che per gli agronomi.

1.2 Diagramma E-R

La generazione delle Dashboard è partita da un diagramma Entità-Relazione, utilizzato come base per definire le principali entità del sistema e le relazioni presenti tra di loro. Questo diagramma è stato fornito all'intelligenza artificiale insieme ad altre istruzioni con vari prompt.

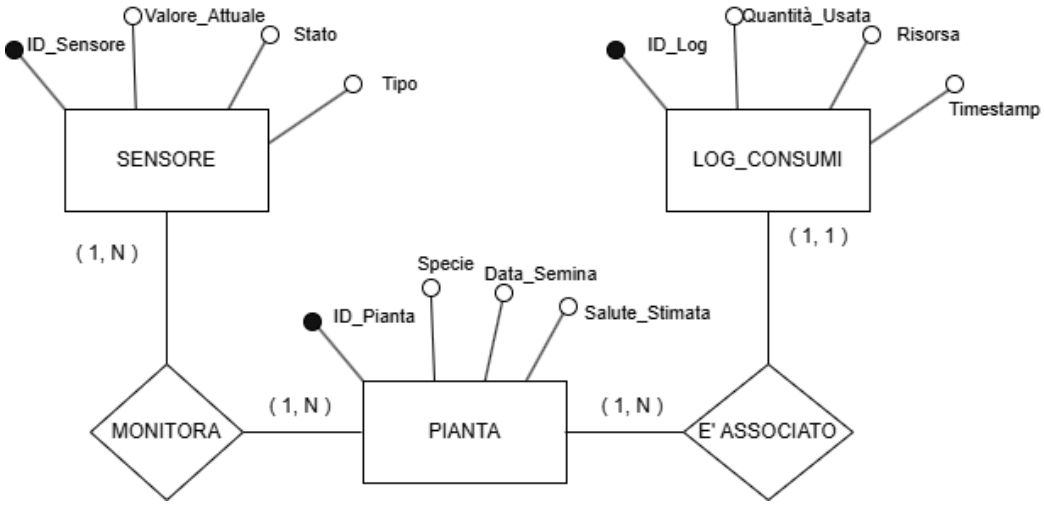


Figura 1: Diagramma E-R

Il diagramma è composto da tre entità:

- Sensore: Rappresenta i vari sensori installati nella GreenHouse, tra i vari attributi troviamo il tipo quindi se misura la Luce, l'Umidità o la Temperatura, il valore attuale, e lo stato, ovvero se è attivo, in manutenzione oppure offline
- Pianta: Rappresenta le piante presenti all'intero della serra, di cui conosciamo la specie, la data in cui è stata piantata e lo stato di salute attuale
- Log_Consumi: Registra l'utilizzo effettivo delle risorse critiche, come l'acqua e l'elettricità, in precisi momenti temporali.

Nel diagramma sono anche definite due relazioni:

- Monitora: Collega i sensori alle piante. Indica che ogni sensore è responsabile del controllo delle condizioni vitali delle piante coltivate. Un sensore monitora più piante, una pianta può essere monitorata da più sensori
- È Associato: Lega le piante ai consumi. Questa connessione permette di capire quanta acqua o energia è stata necessaria per far crescere una specifica pianta o un gruppo di esse. Una pianta è associata a più log dei consumi, un log può essere associato ad una sola pianta.

1.3 Scelta del modello

Per la generazione delle dashboard è stato utilizzato Gemini Nano Banana, è un modello di generazione immagini AI di Google noto per la sua velocità estrema e alta qualità, ideale per iterazioni rapide e bozze veloci. I suoi principali vantaggi includono l'ottima gestione del testo in italiano e delle proporzioni delle immagini.

2 Generazione delle Dashboard

In questa sezione, esploreremo il processo di creazione della dashboard attraverso una sequenza di prompt consecutivi. Per ogni prompt, presenteremo il testo utilizzato, l'immagine generata dal modello e un'analisi del risultato ottenuto. Questo approccio ci permetterà di mostrare come la dashboard sia stata sviluppata attraverso raffinamenti graduali, evidenziando sia i successi che le difficoltà e i limiti riscontrati con il modello generativo.

2.1 Prompt0: Diagramma E-R

Prompt:

Supponiamo di avere il seguente schema logico di una Smart Greenhouse (serra idroponica).

- Sensore: ID_Sensore (PK), Tipo (Luce, Umidità, Temperatura), Valore_Attuale, Stato (Attivo / Manutenzione)
- Pianta: ID_Pianta (PK), Specie, Data_Semina, Salute_Stimata (%)
- Log_Consumi: ID_Log (PK), Risorsa (Acqua, Energia), Quantita_Usata, Timestamp
- Relazioni:
  - Un Sensore monitora più Piante.
  - Una Pianta è associata a più Log_Consumi.

D'ora in avanti ti chiederò di creare immagini di dashboard aziendali basate su questo schema logico.

Osservazioni: Il primo prompt non ha generato alcuna immagine, è stato semplicemente fornito il diagramma E-R all'intelligenza artificiale su cui si baserà per la generazione delle dashboard.

2.2 Prompt1: Generazione prima Dashboard

Prompt:

Crea una dashboard riguardante i sensori della Smart Greenhouse, in modo coerente rispetto allo schema logico fornito, con una prospettiva frontale e senza sfondo

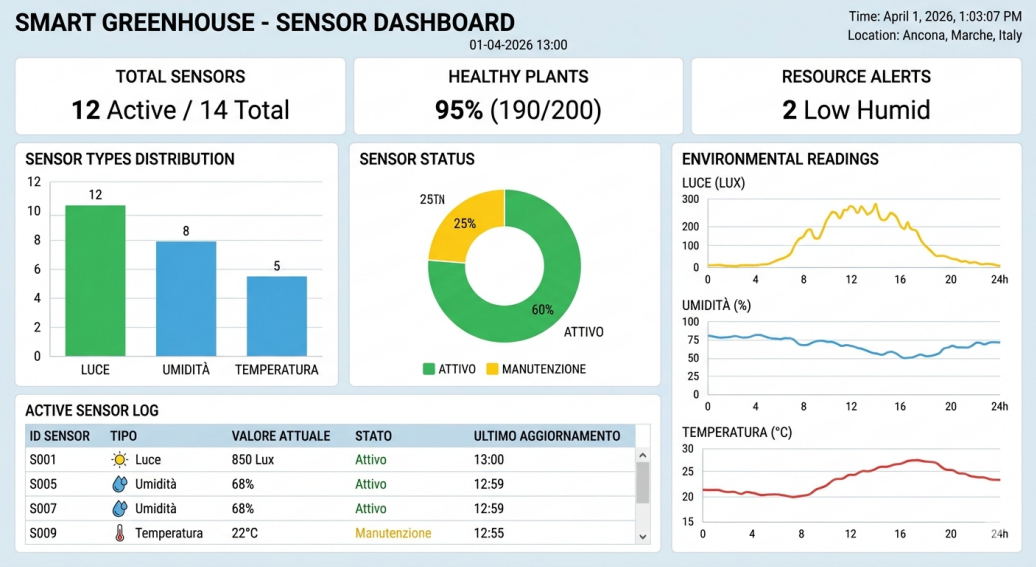


Figura 2: Dashboard generata con il primo prompt da Nano Banana 2

Osservazioni: Il modello ha generato una dashboard semplice e pulita, sono presenti alcuni errori nei grafici e negli indicatori numerici, il grafico a torta non rappresenta il 100% dei sensori mentre il contatore del numero totale dei sensori non è coerente con l'istogramma.

2.3 Prompt2: Modifiche estetiche

Prompt:

Il colore deve essere variato per attirare maggiormente l'attenzione. La dashboard deve mostrare i dati dei sensori nelle ultime 24 ore. La lingua deve essere coerente, rendi tutta la dashboard in italiano Il grafico a torta è sbagliato, non raggiunge il 100% e le etichette sono errate

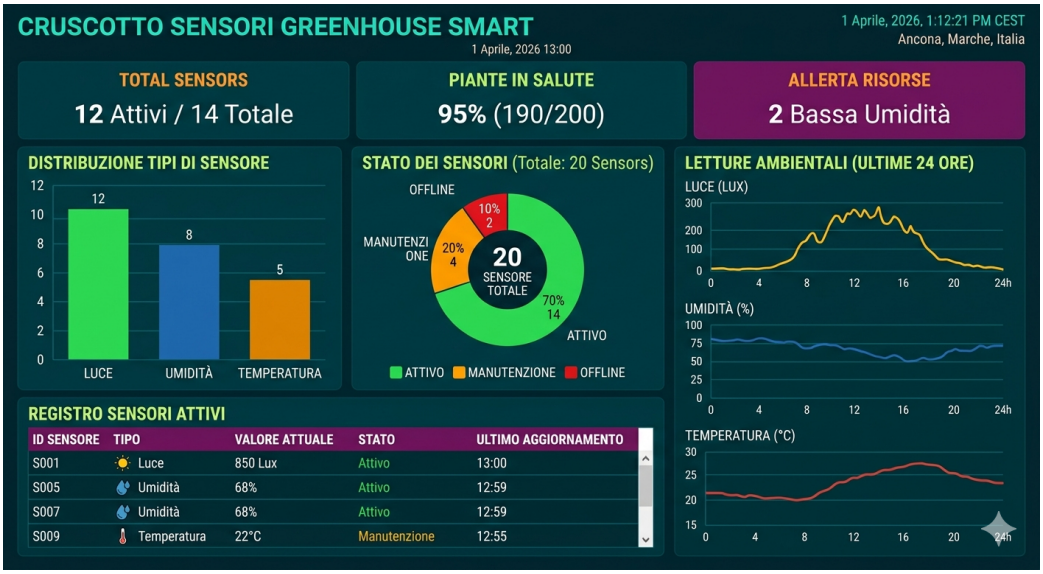


Figura 3: Immagine generata con il secondo prompt da Nano Banana 2

Osservazioni: Con questo prompt il modello ha reso la dashboard più interessante modificandone i colori e rendendola di fatto più accattivante, tuttavia sono ancora presenti errori all'interno dei grafici, tra il grafico a torta, l'istogramma e il contatore c'è ancora incoerenza sul numero totale di sensori.

2.4 Prompt3: Correzione numero sensori

Prompt:

il numero totale di sensori non è coerente nei vari grafici, nel grafico a torta sono indicati 20 sensori, nell'istogramma 25 e nell'indicatore iniziale 14 La lingua non è ancora coerente, ci sono ancora alcune parole in inglese l'indicatore "allerta risorse" non è chiaro a cosa si riferisce

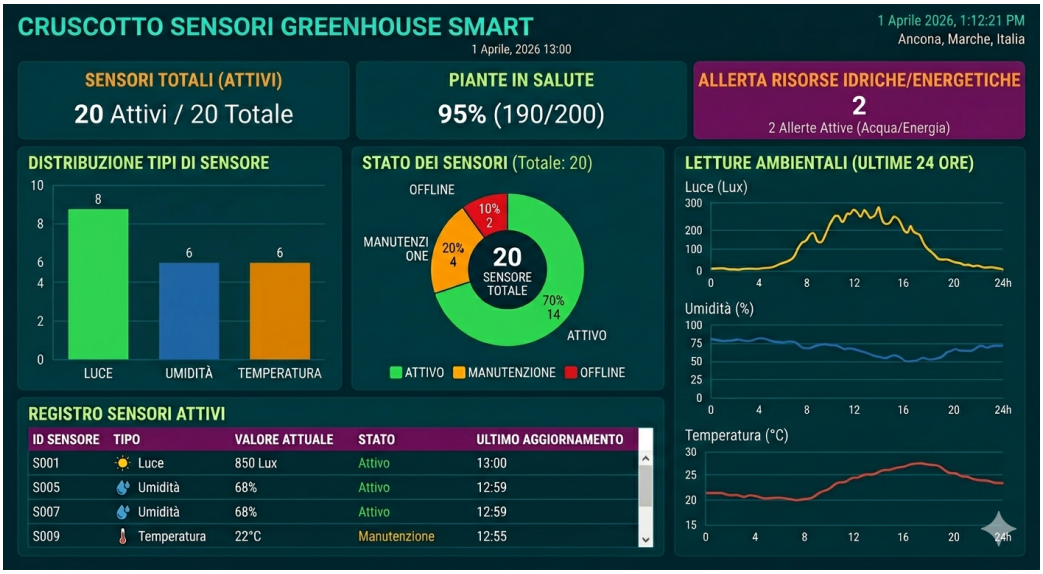


Figura 4: Immagine generata con il terzo prompt da Nano Banana 2

Osservazioni: L'obiettivo principale di questo prompt era quello di rendere coerenti i grafici tra loro e di tradurre la dashboard interamente in italiano. Il testo è stato tradotto in italiano ma i grafici sono ancora errati, il grafico a torta contiene troppe etichette mentre il numero totale di sensori attivi è errato.

2.5 Prompt4: Ricorrezione numero sensori e correzione grafici

Prompt:

Il numero totale di sensori attivi è errato dal grafico a torta si evince che 2 sensori sono offline e 4 in manutenzione Migliora le etichette del grafico a torta Rimuovi la parola attivi dal titolo "registro sensori attivi" visto che sono presenti anche sensori in manutenzione

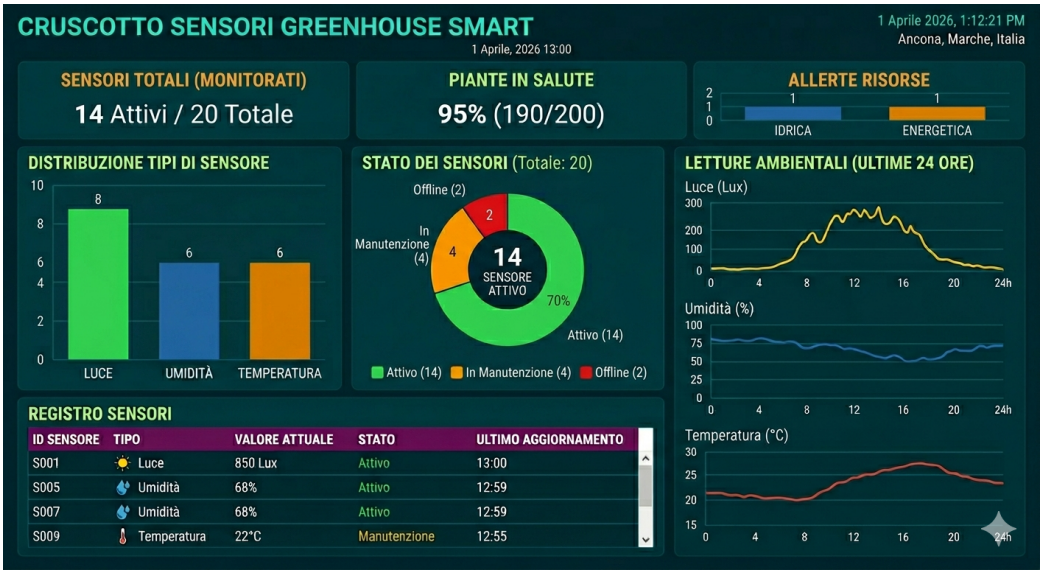


Figura 5: Immaigne generata con il terzo prompt da Nano Banana 2

Osservazioni: Il numero dei sensori ora è coerente tra i vari grafici ed è stato corretto il titolo del grafico relativo allo stato dei sensori.

2.6 Prompt5: Modifica grafici

Prompt:

Per l'allerta delle risorse non utilizzerei un istogramma, ma un altro tipo di indicatore, ad esempio un manometro. Il grafico a torta deve mostrare o le percentuali o il numero, non entrambi, e rimuovi il numero di elementi vicino al nome dell'etichetta

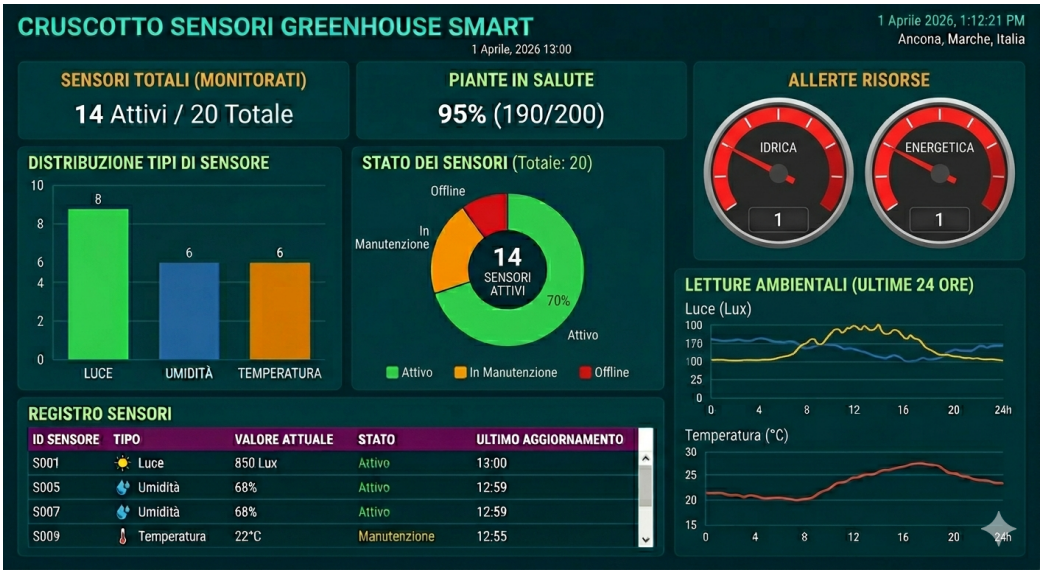


Figura 6: Immagine generata con il quinto prompt da Nano Banana 2

Osservazioni: Il grafico delle allerte per le risorse è stato modificato come da richiesta, anche se occupa troppo spazio, infatti è stato rimosso il grafico relativo alla Luce nelle letture ambientali. Inoltre non tutti gli spicchi del grafico a torta contengono le percentuali che rappresentano.

2.7 Prompt6: Modifica grafico a torta

Prompt:

Inserisci la percentuale in tutti gli spicchi del grafico a torta Rimpicciolisci gli indicatori delle allerte e ripristina i grafici delle letture ambientali

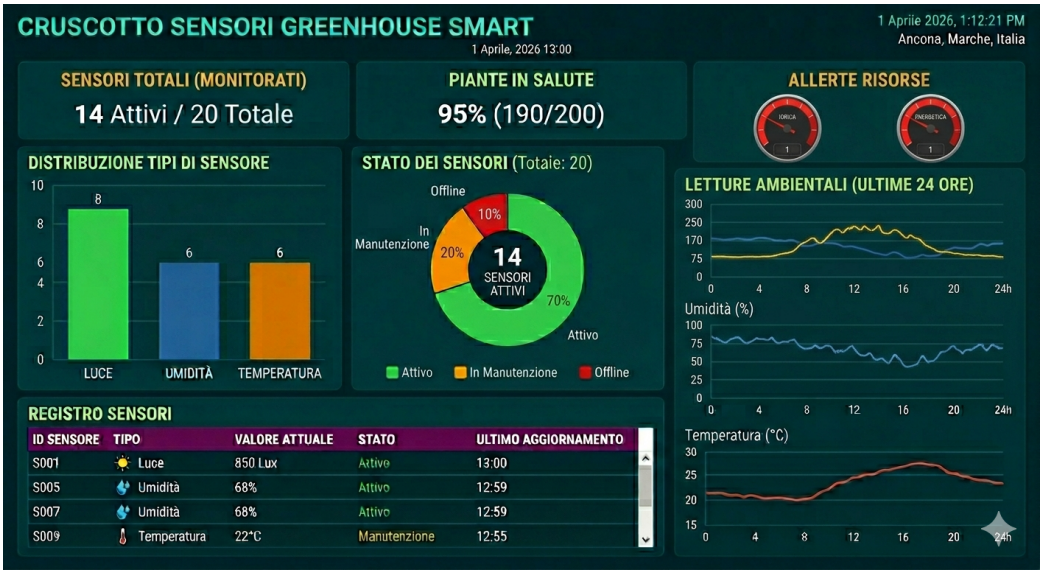


Figura 7: Immagine generata con il sesto prompt da Nano Banana 2

Osservazioni: Le percentuali del grafico a torta sono ora corrette ed è stato ripristinato il gradico delle letture della Luce, tuttavia il grafico della luce ora è errato poichè contiene due curve.

2.8 Prompt7: Ripristino grafico letture ambientali

Prompt:

reinserisci il titolo nel grafico della luce (Lux) e rimuovi la curva blu dal grafico della luce

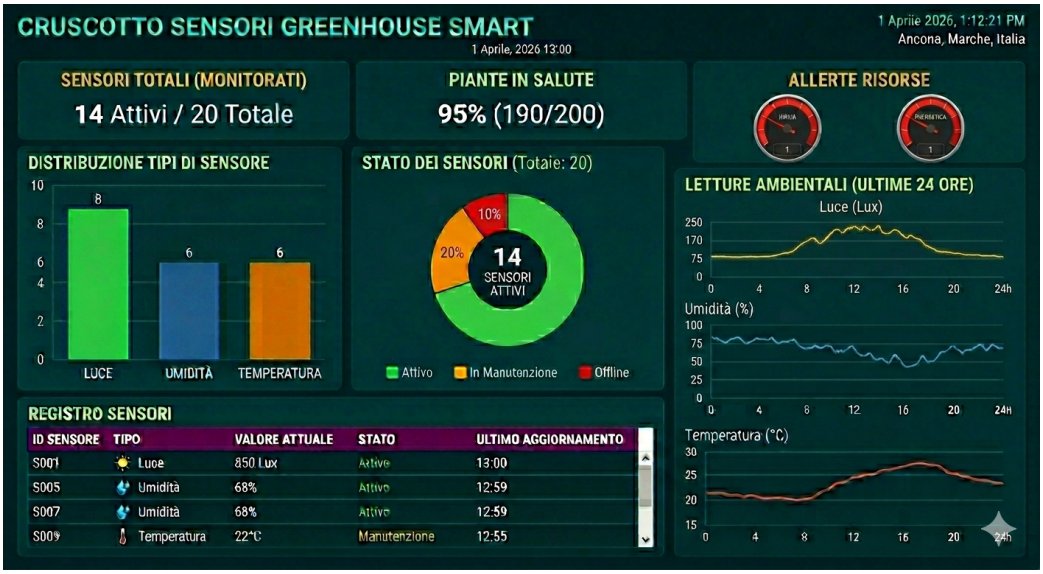


Figura 8: Immagine generata con il settimo prompt da Nano Banan 2

Osservazioni: I grafici sono stati ripristinati e sono coerenti tra di loro, il risultato ottenuto è accettabile.

2.9 Dashboard 2

2.9.1 Prompt1:

Prompt:

Creiamo una nuova dashboard: Crea un'immagine ad alta risoluzione (1920x1080) di una dashboard professionale intitolata "Dashboard Piante – Smart Greenhouse", progettata da un data analyst. La dashboard deve basarsi ESCLUSIVAMENTE sullo schema logico fornito e NON deve introdurre nuovi attributi.

- Entità principale: PIANTA
- Attributi disponibili: ID_Pianta, Specie, Data_Semina, Salute_Stimata
- Relazione: Una Pianta è associata a più Log_Consumi.
- Stile grafico:
  - design moderno e pulito

  - sfondo scuro tecnologico
  - palette colori: verde brillante, blu acqua, giallo luce
  - font sans-serif leggibile
  - box con angoli arrotondati
  - layout ordinato a griglia

Layout:

- HEADER Titolo grande: Dashboard Piante – Smart Greenhouse
- Sottotitolo: Monitoraggio salute e crescita delle colture idroponiche
- KPI in alto (4 box):
  - Numero totale piante
  - Salute media delle piante (%)
  - Numero medio di giorni dalla semina
  - Pianta con salute più alta (Specie)
- GRAFICO 1, Grafico a barre: "Numero di piante per specie", con asse X: specie delle piante, e asse Y: numero di piante. Ogni barra deve avere il valore numerico sopra.
- GRAFICO 2, Grafico a barre: "Salute media per specie" con asse X: specie, e asse Y: percentuale di salute.
- GRAFICO 3, Grafico timeline: "Date di semina", mostra le piante distribuite nel tempo in base alla Data_Semina.
- TABELLA: Tabella leggibile con 6–8 righe fittizie. Colonne:
  - ID_Pianta
  - Specie
  - Data_Semina
  - Salute_Stimata

Assicurati che:

- tutti i testi siano completamente leggibili
- non vengano tagliate le parole

- tutte le etichette siano in lingua italiana
- non usare testo generico come "Lorem ipsum"

Mostra la dashboard completa in un'unica schermata, come screenshot di un software di business intelligence.

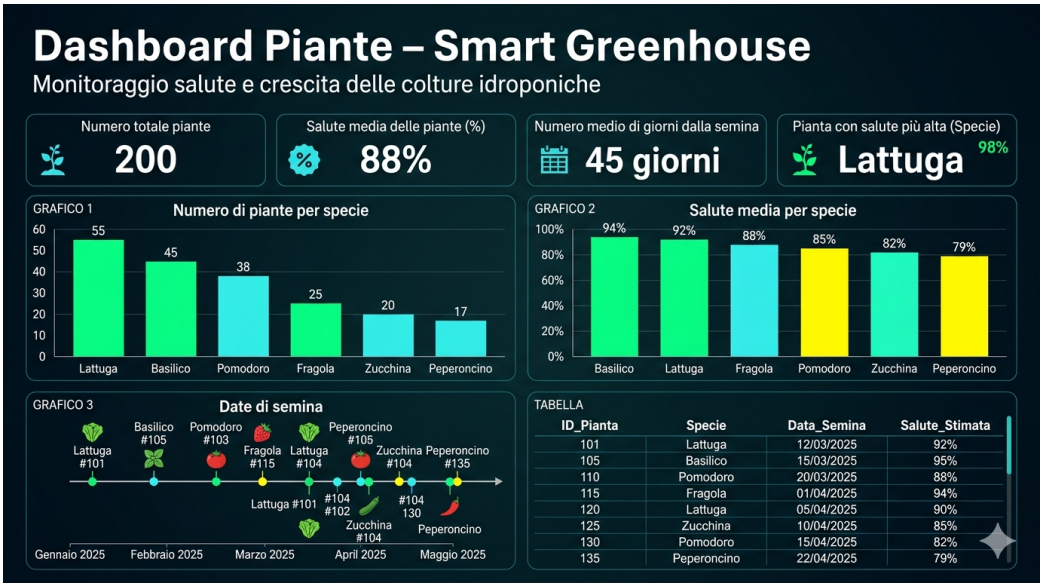


Figura 9: Immagine generata con il primo prompt per la seconda dashboard da Nano Banana 2

Osservazioni: Il prompt ha generato un'interessante dashboard sulle piante coltivate all'interno della serra. Sono però presenti degli errori e delle incoerenze tra i vari grafici, specialmente sul grafico della salute media per specie, infatti indica che la pianta con la salute media più alta è il Basilico mentre dagli altri grafici si evince che è la Lattuga.

2.9.2 Prompt2 Dashboard 2

Prompt:

La dashboard contiene alcuni errori:

- alcuni valori percentuali non sono coerenti con i grafici
- la specie con salute più alta nei KPI non coincide con il grafico

Mantieni la dashboard invariata ma correggi questi errori:

- elimina errori grammaticali
- rendi coerenti i valori numerici
- assicurati che tutti i grafici abbiano un titolo
- verifica la coerenza tra KPI e grafici.

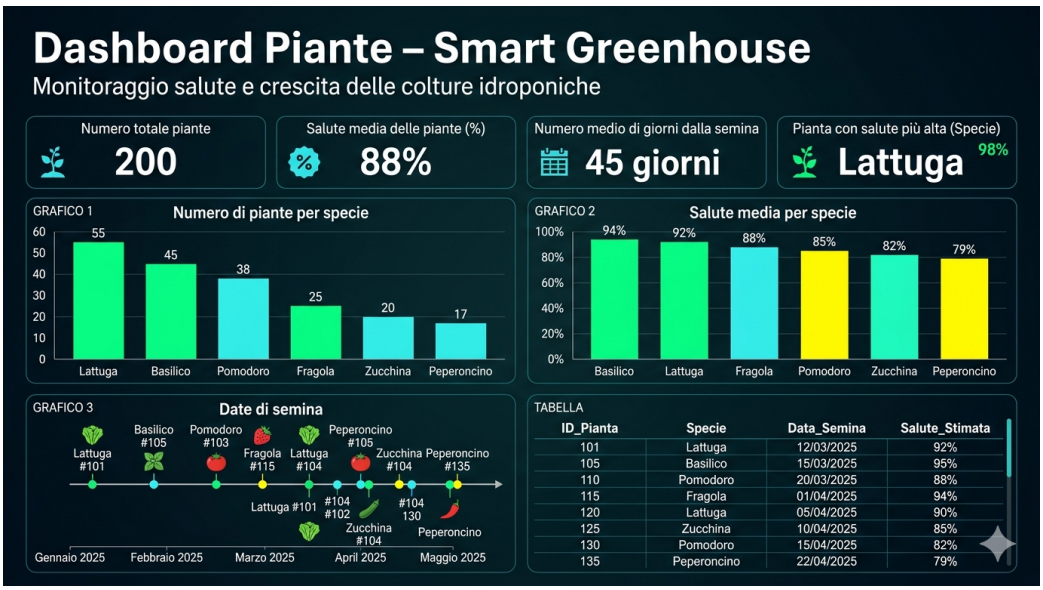


Figura 10: Immagine generata con il secondo prompt per la seconda dashboard da Nano Banana 2

Osservazioni: Tutte le correzioni richieste dal prompt sono state soddisfatte, ora i grafici sono corretti e coerenti tra di loro. Il risultato finale è una dashboard semplice ma d'impatto sullo stato di salute generale delle piante coltivate all'interno della greenhouse.

3 Simulatore di Epidemie

Questo progetto consiste nello sviluppo di un simulatore interattivo di diffusione di un'epidemia, realizzato tramite un programma con interfaccia grafica. L'obiettivo del software è permettere all'utente di osservare e comprendere in modo intuitivo come una malattia possa diffondersi all'interno di una popolazione e come diversi fattori possano influenzarne l'andamento. Attraverso la simulazione è possibile visualizzare il comportamento dinamico di individui che si muovono nello spazio e che possono contagiare altre persone in base a determinate probabilità. Il progetto è stato sviluppato utilizzando il linguaggio di programmazione Python e una libreria per la creazione dell'interfaccia grafica, in modo da rendere la simulazione facilmente comprensibile e interattiva per l'utente.

3.1 Descrizione del software

Il software sviluppato è un simulatore epidemiologico semplificato che rappresenta una popolazione tramite punti che si muovono all'interno di uno spazio grafico. Ogni punto rappresenta una persona e può trovarsi in diversi stati: sano, infetto, guarito, vaccinato oppure deceduto. Durante la simulazione le persone si muovono casualmente e, quando una persona infetta si avvicina a una sana, esiste una probabilità che avvenga il contagio.

L'utente può configurare diversi parametri prima di avviare la simulazione, tra cui:

- numero di persone nella popolazione
- probabilità di contagio
- tempo necessario per la guarigione
- tasso di mortalità
- percentuale di persone vaccinate
- efficacia delle cure

Durante l'esecuzione del programma vengono mostrati indicatori e statistiche in tempo reale, come il numero di persone sane, infette, guarite, vaccinate e decedute. Inoltre è possibile visualizzare l'andamento dell'epidemia nel tempo tramite grafici o contatori aggiornati dinamicamente.

3.2 Scelta del modello

Per la realizzazione del progetto è stato utilizzato Gemini 1.5 Flash, un modello di intelligenza artificiale progettato per supportare la generazione e l'assistenza nella scrittura di codice. Questo strumento è stato scelto perché permette di generare rapidamente esempi di codice funzionanti, fornire spiegazioni sui concetti utilizzati e aiutare nel miglioramento progressivo del programma.

L'utilizzo di Gemini 1.5 Flash ha consentito di velocizzare il processo di sviluppo, permettendo di concentrarsi maggiormente sulla progettazione della simulazione e sull'aggiunta di nuove funzionalità. Inoltre lo strumento ha facilitato la comprensione di alcune parti del codice e ha supportato l'implementazione di diverse caratteristiche avanzate del software.

4 Processo di generazione

Lo sviluppo del progetto è avvenuto in modo progressivo e iterativo. Inizialmente è stata realizzata una versione molto semplice del simulatore, che prevedeva solo la presenza di persone sane e infette e un meccanismo base di contagio. Successivamente il programma è stato migliorato aggiungendo gradualmente nuove funzionalità.

Le principali fasi di sviluppo sono state:

1. creazione della struttura base del programma con interfaccia grafica;
2. implementazione della popolazione e del movimento casuale delle persone;
3. introduzione del meccanismo di contagio tra individui;
4. aggiunta del sistema di guarigione dopo un certo periodo di tempo;
5. introduzione di nuovi stati come vaccinati e deceduti;
6. aggiunta di parametri configurabili dall'utente;
7. inserimento di contatori e statistiche per monitorare l'andamento dell'epidemia.

Durante ogni fase il codice è stato migliorato e testato per verificare il corretto funzionamento della simulazione.

4.1 Prompt1: Simulatore base

Prompt:

Scrivi un programma in Python con interfaccia grafica usando Tkinter che simuli la diffusione di un'epidemia.

Requisiti:

- creare una finestra con un canvas che rappresenta l'ambiente
- generare una popolazione di punti che rappresentano le persone
- ogni persona può avere uno stato: sano (verde), infetto (rosso) e guarito (blu)

Regole della simulazione:

- le persone si muovono casualmente nello spazio
- quando una persona infetta si avvicina a una persona sana può contagiarla
- dopo un certo tempo una persona infetta guarisce

Interfaccia utente:

- campo per inserire la popolazione
- campo per inserire la probabilità di contagio
- campo per inserire il tempo di guarigione
- pulsante "Avvia simulazione"

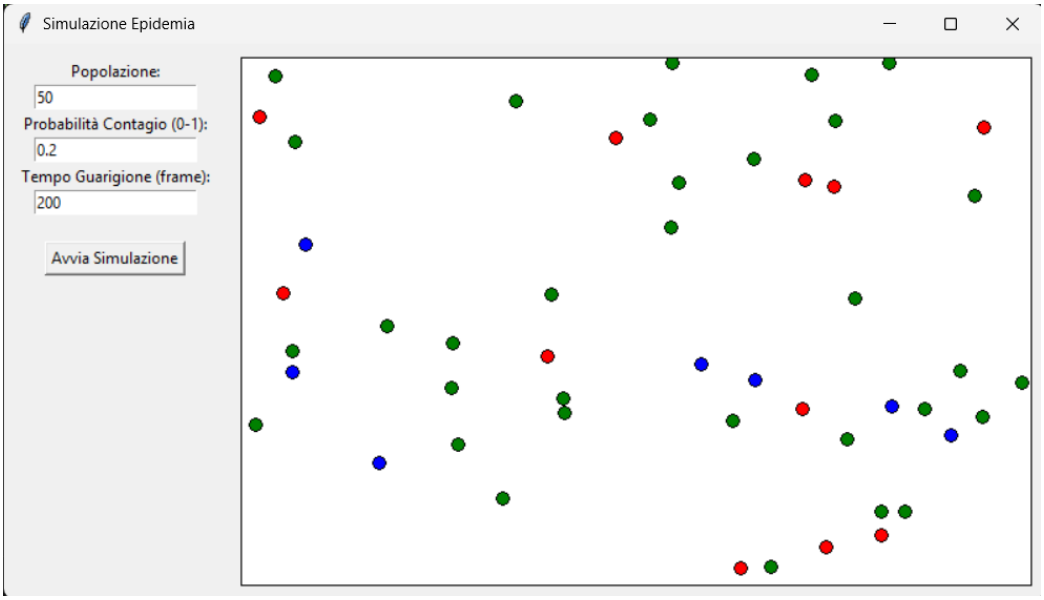


Figura 11: Prima versione del software generato da Gemini 1.5 Flash

Osservazioni: Il primo risultato è coerente con le richieste definite nel prompt iniziale e implementa correttamente le funzionalità di base del simulatore. Tuttavia, presenta una limitazione: la simulazione non prevede una condizione di arresto e continua a essere eseguita anche quando non sono più presenti individui infetti, rendendo l'esecuzione potenzialmente infinita.

4.2 Prompt2: Migliorare la GUI

Prompt:

Migliora l'interfaccia grafica del simulatore di epidemia scritto in Python con Tkinter.

Aggiungi:

- un layout ordinato con frame
- slider per inserire i parametri invece dei campi di testo
- etichette chiare per ogni parametro

Aggiungi anche contatori in tempo reale che mostrino:

- numero di persone sane
- numero di persone infette
- numero di persone guarite

I contatori devono aggiornarsi durante la simulazione. Quando non ci sono più infetti terminare la simulazione.

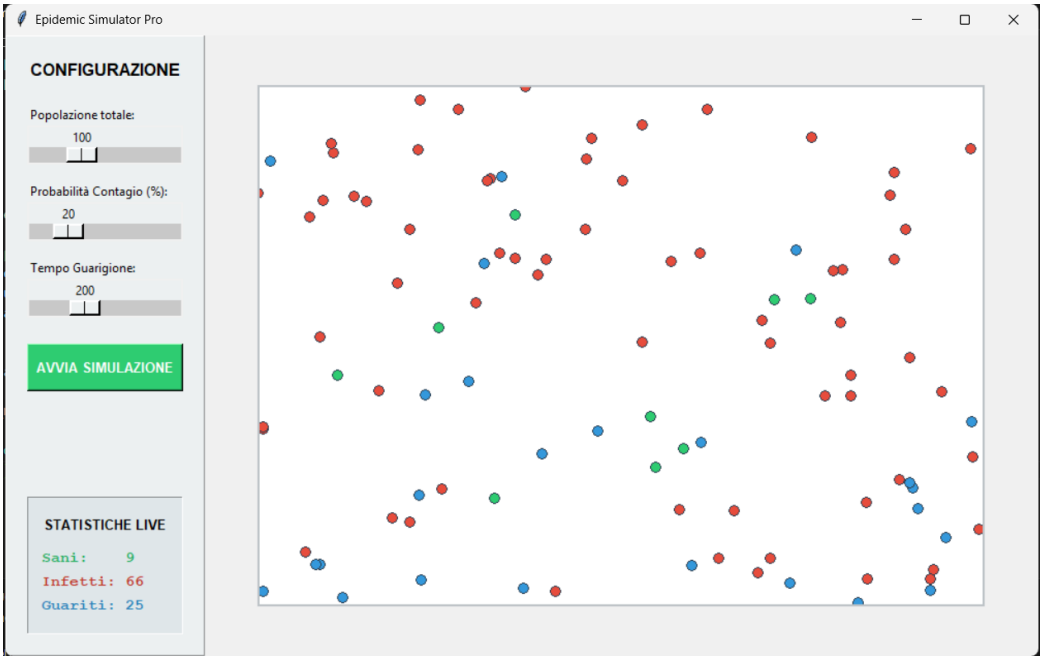


Figura 12: Seconda versione del software generato da Gemini 1.5 Flash

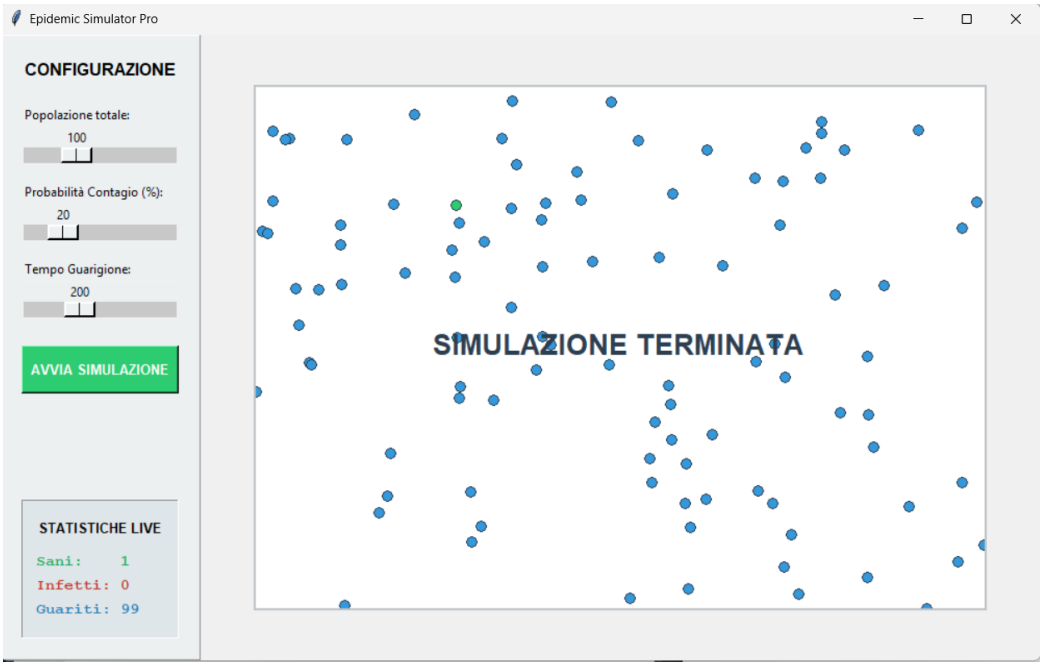


Figura 13: Termine della seconda versione del software generato da Gemini 1.5 Flash

Osservazioni: Il secondo risultato soddisfa le richieste del prompt, presentando un'interfaccia grafica più organizzata e intuitiva. In particolare, l'introduzione di un layout strutturato, l'utilizzo degli slider per l'inserimento dei parametri e la presenza di contatori aggiornati in tempo reale rendono il software più usabile e visivamente più chiaro. Tuttavia, nonostante questi miglioramenti, il programma presenta ancora margini di ottimizzazione, sia dal punto di vista estetico sia in termini di funzionalità e completezza della simulazione.

4.3 Prompt3: Grafico dell'epidemia

Prompt:

Aggiungi al simulatore di epidemia un grafico che mostri l'andamento della simulazione nel tempo.

Requisiti:

- utilizzare matplotlib
- integrare il grafico nella finestra Tkinter
- il grafico deve mostrare tre linee: sani, infetti, guariti
- i dati devono aggiornarsi durante la simulazione

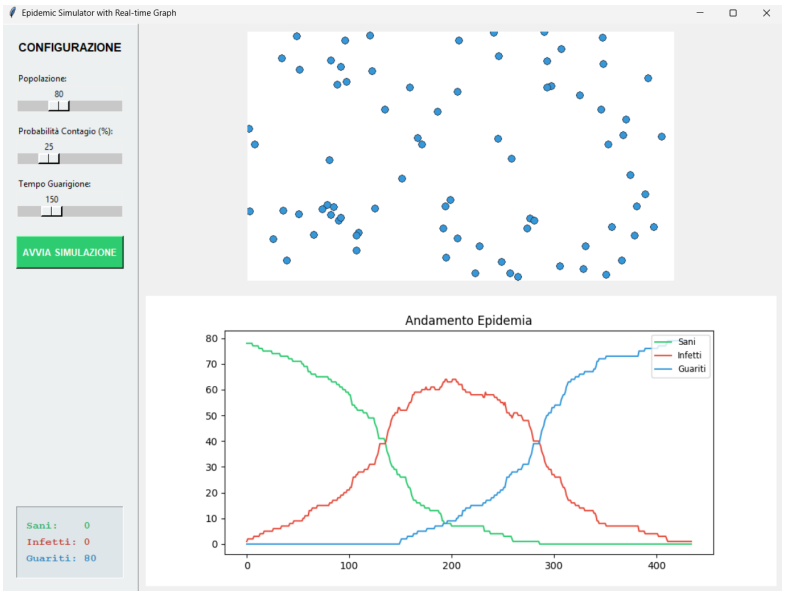


Figura 14: Terza versione del software generato da Gemini 1.5 Flash

Osservazioni: Il terzo risultato implementa correttamente tutte le modifiche richieste dal prompt, in particolare l'aggiunta del grafico che mostra l'andamento della simulazione nel tempo e il suo aggiornamento dinamico. Tuttavia, a differenza della versione precedente, il programma non segnala in alcun modo la conclusione della simulazione, risultando quindi meno chiaro per l'utente nel momento in cui l'epidemia termina, nonostante tutte le funzionalità richieste siano state effettivamente implementate.

4.4 Prompt4: Aggiungere mortalità

Prompt:

Estendi il simulatore introducendo il tasso di mortalità. Nuovo stato: deceduto (nero)

Regole:

- quando una persona infetta supera il tempo di malattia può guarire oppure decedere
- la probabilità di decedere dipende dal tasso di mortalità impostato dall'utente

Interfaccia:

- aggiungere uno slider per il tasso di mortalità
- aggiungere un contatore per il numero di morti
- aggiornare il grafico per includere anche la curva dei deceduti

Indicare quando la simulazione è terminata.

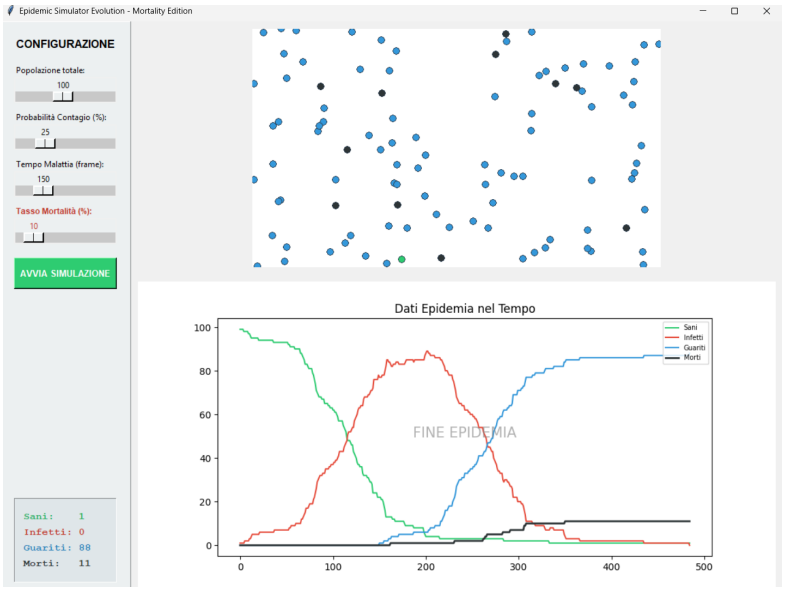


Figura 15: Quarta versione del software generato da Gemini 1.5 Flash

Osservazioni: Il quarto risultato soddisfa pienamente tutte le richieste del prompt. Le nuove funzionalità sono state implementate correttamente e integrate in modo coerente nel simulatore, senza introdurre problemi evidenti. Nel complesso, l'esecuzione risulta completa e funzionante.

4.5 Prompt5: Vaccinazione

Prompt:

Aggiungi al simulatore la vaccinazione. Nuovo stato: vaccinato (giallo)
Funzionamento:

- l'utente può scegliere la percentuale iniziale di popolazione vaccinata
- i vaccinati hanno una probabilità molto più bassa di essere contagiati
- aggiungere un parametro per l'efficacia del vaccino

Interfaccia:

- slider per percentuale vaccinati iniziali
- slider per efficacia vaccino
- contatore del numero di vaccinati

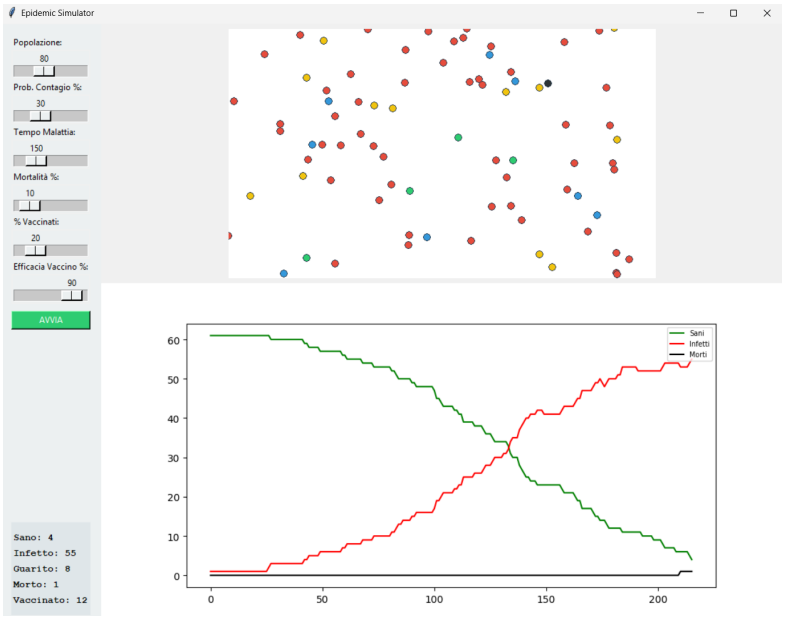


Figura 16: Quinta versione del software generato da Gemini 1.5 Flash

Osservazioni: Inizialmente la simulazione non si avviava a causa di un errore nel codice. In particolare, si verificava un errore di tipo `KeyError: 'sano'`, dovuto a una discrepanza tra i nomi delle chiavi utilizzate nel dizionario delle statistiche (`self.stats_labels`) e quelle impiegate nel conteggio

interno della simulazione (`counts`). Nello specifico, nella definizione delle etichette era stato utilizzato il termine "Sani" (plurale), mentre nella logica di calcolo veniva utilizzato "sano" (singolare), causando così il malfunzionamento. Successivamente è stato quindi inviato un ulteriore prompt per ottenere una versione funzionante del simulatore.

Prompt extra: Nel codice è presente il seguente problema: Exception in Tkinter callback, KeyError: 'sano'

Risultato: Grazie a questo prompt il modello ha restituito un codice eseguibile e con le modifiche richieste, aggiungendo uno slider per la percentuale dei vaccinati iniziali, uno slider per l'efficacia del vaccino ed è stato aggiunto un contatore per il numero di vaccinati.

4.6 Prompt6: Cure

Prompt:

Aggiungi al simulatore un sistema di cure.
Funzionamento:

- l'utente può impostare l'efficacia delle cure
- le cure riducono il tasso di mortalità degli infetti

Regola: mortalità_effettiva = mortalità * (1 - efficacia_cure)
Interfaccia:

- slider per efficacia delle cure
- aggiornare le statistiche finali della simulazione

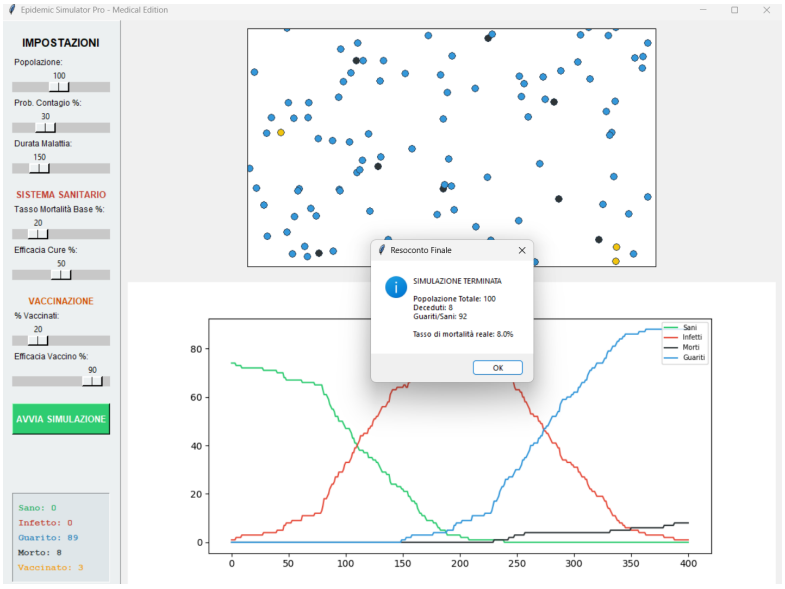


Figura 17: Sesta versione del software generato da Gemini 1.5 Flash

Osservazioni: Il sesto risultato ha soddisfatto le richieste del prompt, implementando correttamente le funzionalità previste. Inoltre, ha introdotto miglioramenti aggiuntivi non esplicitamente richiesti: in particolare ha modificato la grafica degli indicatori, rendendoli più chiari e distinguibili, e ha aggiunto autonomamente un report finale della simulazione, arricchendo ulteriormente il software dal punto di vista funzionale e informativo.

4.7 Prompt7: Controlli della simulazione

Prompt:

Aggiungi controlli completi per la simulazione.
La GUI deve includere:

- pulsante pausa
- pulsante riprendi
- pulsante reset
- pulstante nuova simulazione

Il reset deve riportare la simulazione allo stato iniziale con gli stessi parametri.

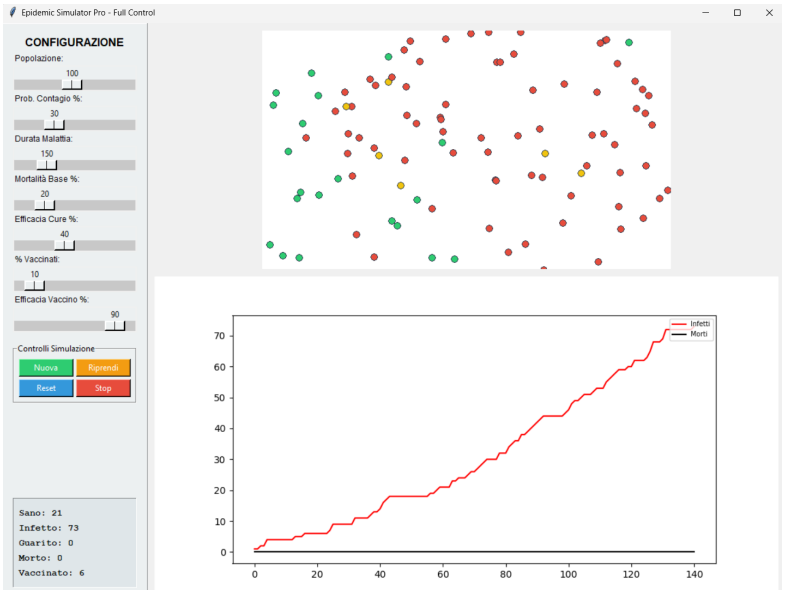


Figura 18: Settima versione del software generato da Gemini 1.5 Flash

Osservazioni: Il settimo risultato ha soddisfatto le richieste del prompt, introducendo correttamente i controlli per la gestione della simulazione, tra cui i pulsanti per mettere in pausa, fermare e resettare l'esecuzione, tutti pienamente funzionanti. Tuttavia, come effetto collaterale, sono state rimosse alcune curve dal grafico dell'andamento dell'epidemia, riducendo la completezza delle informazioni visualizzate.

4.8 Prompt8: Statistiche finali

Prompt:

Aggiungi una schermata di riepilogo quando l'epidemia termina.
Mostra:

- numero totale di persone contagiate
- numero totale di morti
- numero totale di guariti
- numero totale di infetti
- durata dell'epidemia

Mostra queste informazioni in una finestra di riepilogo.



Figura 19: Ottava versione del software generato da Gemini 1.5 Flash

Osservazioni: In questa versione è stato rimosso il tasto di stop, cosa non richiesta. Inoltre, inizialmente la simulazione non si avviava a causa di un errore, l'errore *AttributeError: 'SimulationApp' object has no attribute 'time_steps'* indica che il programma sta cercando di aggiungere dati a una lista (time_steps) che non è stata creata correttamente nel momento in cui la simulazione parte. Successivamente 'e stato quindi inviato un ulteriore

prompt per ottenere una versione funzionante del simulatore.

Prompt extra: Nel codice è presente il seguente errore: AttributeError: 'SimulationApp' object has no attribute 'time_stamps'

Risultato: Grazie a questo prompt il modello ha restituito un codice eseguibile e con le modifiche richieste, aggiungendo una schermata di ripeilogo al termine dell'epidemia

4.9 Prompt9: Migliorare l'animazione

Prompt:

Migliora la simulazione rendendo il movimento delle persone più fluido.
Aggiungi:

- regolazione della velocità della simulazione
- movimento più realistico delle persone
- possibilità di cambiare dimensioni della popolazione senza rallentare troppo la simulazione

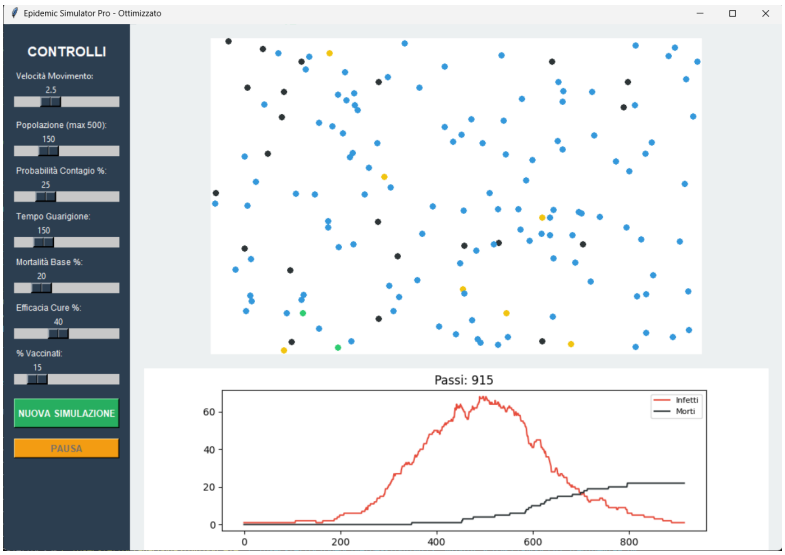


Figura 20: Nona versione del software generato da Gemini 1.5 Flash

Osservazioni: Nel nono risultato sono state apportate alcune modifiche all'interfaccia e al funzionamento della simulazione. La qualità complessiva del programma è migliorata: la simulazione risulta più fluida e realistica, e anche l'interfaccia di controllo è stata resa più efficace e intuitiva per l'utente. Tuttavia, sono stati rimossi il tasto di reset e il report finale della simulazione, modifiche che non erano desiderate e che riducono la completezza del software.

4.10 Prompt10: Dashboard finale

Prompt:

Trasforma il simulatore in una applicazione completa con una dashboard.
La finestra deve essere divisa in:

- pannello di controllo
- area di simulazione
- grafico dell'epidemia
- pannello con statistiche in tempo reale

Usa un layout professionale con Tkinter o CustomTkinter.

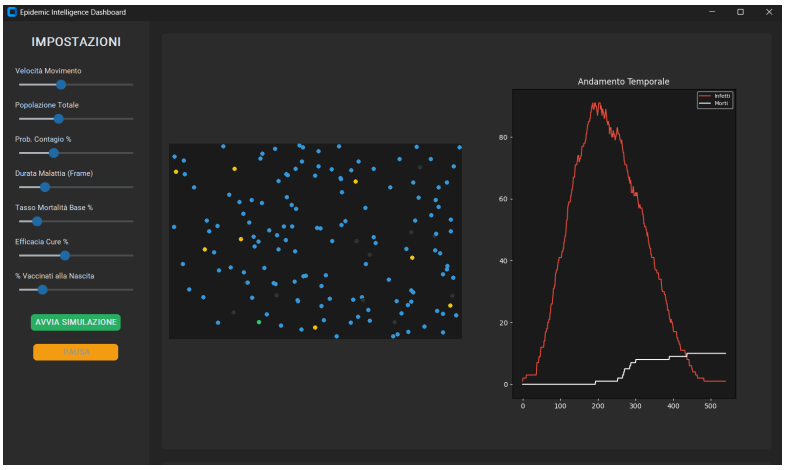


Figura 21: Decima versione del software generato da Gemini 1.5 Flash

Osservazioni: Il decimo risultato rappresenta un buon avanzamento del progetto, con una simulazione più completa e un'interfaccia curata. Tuttavia, il grafico dell'andamento dell'epidemia non mostra ancora tutte le curve previste e manca il report finale della simulazione, limitando la piena fruibilità delle informazioni per l'utente.

4.11 Prompt11: Correzioni finali

Prompt:

Riaggiungi il report finale alla fine della simulazione, con statistiche dettagliate su contagiati, guariti, deceduti e durata dell'epidemia. Nel grafico inserisci anche la curva che rappresenta i sani, in modo da avere una visione completa dell'andamento dell'epidemia.



Figura 22: Undicesima versione del software generato da Gemini 1.5 Flash

Osservazioni: L'ultimo prompt ha portato il software a una soluzione completa e soddisfacente, con tutte le funzionalità principali integrate: simulazione fluida, interfaccia di controllo completa, gestione dei parametri, visualizzazione degli stati della popolazione e aggiornamento dei contatori in tempo reale. Inoltre, il grafico dell'andamento dell'epidemia è ora completo e il report finale è incluso.

5 Risultati

L'obiettivo del progetto era quello di sviluppare due use case utilizzando l'intelligenza artificiale generativa, in particolare applicare tecniche di prompt engeneering e di valutare come si comportano questi modelli generativi.

Il risultato finale è un software interattivo con interfaccia grafica che permette di simulare la diffusione di una malattia all'interno di una popolazione virtuale, e delle dashboard per l'analisi dei dati di una serra idroponica intelligente.

Il progetto vuole dimostrare come sia possibile utilizzare strumenti di intelligenza artificiale generativa per scrivere codice al fine di simulare e rappresentare fenomeni complessi in modo semplice e intuitivo e per generare dashboard utili e d'impatto.

5.1 Dashboard

Le dashboard generate a partire dal diagramma Entità-Relazione offrono una visualizzazione chiara e immediata delle metriche chiave della serra. Nello specifico, la prima dashboard si concentra sul monitoraggio delle infrastrutture, analizzando lo stato dei sensori e l'allocazione delle risorse. La seconda, invece, fornisce una panoramica generale dello stato di salute e dello sviluppo delle colture.

5.2 Software

L'utente può modificare diversi parametri della simulazione e osservare in tempo reale come questi influenzano la diffusione dell'epidemia.

Il programma mostra visivamente le persone che si muovono nello spazio e cambiano stato nel tempo, permettendo di comprendere in modo immediato i meccanismi di contagio. Grazie ai contatori e agli indicatori presenti nell'interfaccia, è possibile analizzare l'andamento della simulazione e confrontare i risultati ottenuti con parametri diversi.

Elenco delle figure

1 Diagramma E-R 3
2 Dashboard generata con il primo prompt da Nano Banana 2 . 5
3 Immagine generata con il secondo prompt da Nano Banana 2 . 6
4 Immagine generata con il terzo prompt da Nano Banana 2 . . 7
5 Immaigne generata con il terzo prompt da Nano Banana 2 . . 8
6 Immagine generata con il quinto prompt da Nano Banana 2 . 9
7 Immagine generata con il sesto prompt da Nano Banana 2 . . 10
8 Immagine generata con il settimo prompt da Nano Banan 2 . 11
9 Immagine generata con il primo prompt per la seconda dashboard da Nano Banana 2 13
10 Immagine generata con il secondo prompt per la seconda dashboard da Nano Banana 2 14
11 Prima versione del software generato da Gemini 1.5 Flash . . . 18
12 Seconda versione del software generato da Gemini 1.5 Flash . 19
13 Termine della seconda versione del software generato da Gemini 1.5 Flash 20
14 Terza versione del software generato da Gemini 1.5 Flash . . . 21
15 Quarta versione del software generato da Gemini 1.5 Flash . . 22
16 Quinta versione del software generato da Gemini 1.5 Flash . . 23
17 Sesta versione del software generato da Gemini 1.5 Flash . . . 25
18 Settima versione del software generato da Gemini 1.5 Flash . . 26
19 Ottava versione del software generato da Gemini 1.5 Flash . . 27
20 Nona versione del software generato da Gemini 1.5 Flash . . . 29
21 Decima versione del software generato da Gemini 1.5 Flash . . 30
22 Undicesima versione del software generato da Gemini 1.5 Flash 31